IMPLEMENTACJA WZORCA PRODUCENT - KONSUMENT GENERUJĄCEGO STATYSTYKI SŁÓW Z WYKORZYSTANIEM PROGRAMOWANIA FUNKCYJNEGO I WSPÓŁBIEŻNEGO

Najnowszą wersję oprogramowania wykorzystywanego na zajęciach laboratoryjnych (Eclipse 2024-09, JavaFX SDK 23, Scene Builder) można ściągnąć z

https://utpedupl-my.sharepoint.com/:u:/g/personal/dszczeg_o365_pbs_edu_pl/EfcqMZaTR9pDsIW0xeVeWUIBwE6joSRvknnYAQnTnNQ7vw?e=ERhp8y

1. WPROWADZENIE

A. WYRAŻENIA LAMBDA

Wyrażenia lambda w Javie to sposób definiowania anonimowych funkcji, umożliwiający pisanie krótszego, bardziej zwięzłego kodu. Ponadto lambdy pozwalają na przekazywanie funkcji jako argumentów do innych metod. Poniżej zostały przedstawione dwa przykłady równoważnych fragmentów kodu – jeden za pomocą tradycyjnego podejścia, a drugi przy użyciu wyrażenia lambda.

```
//1. Definiowanie wątku za pomocą klasy anonimowej
Runnable task = new Runnable() {
   @Override
   public void run() {
         System.out.println("Wątek uruchomiony!");
   }
 };

 //2. Definiowanie wątku przy użyciu wyrażenia lambda
 Runnable task2 = () -> { System.out.println("Wątek uruchomiony!"); };
```

Wyrażenia lambda w Javie mogą być stosowane z tzw. interfejsami funkcyjnymi - zawierającymi tylko jedną metodę abstrakcyjną. W powyższym przykładzie wykorzystany został interfejs `Runnable`, który zawiera jedną metodę `void run()`, dlatego też możliwe było użycie wyrażenia lambda.

Aby ułatwić tworzenie i korzystanie z wyrażeń lambda w pakiecie *java.util.function* utworzono zbiór przydatnych interfejsów funkcyjnych (kilka przedstawiono poniżej). Każdy z nich posiada tylko jedną metodę abstrakcyjną, której konkretną implementację dostarcza tworzone przez programistę wyrażenie lambda.

- `Predicate<T>` - reprezentuje tzw. predykat, czyli funkcję, która zwraca wartość logiczną i przyjmuje jeden argument tj. `boolean test(T t)`.
- `Consumer<T>` - reprezentuje operację, która przyjmuje jeden argument i nic nie zwraca tj. `void accept(T t)`.
- `Function<T, R>` - reprezentuje funkcję, która przekształca obiekt typu T na obiekt typu R tj. `R apply(T t)`.
- `Supplier<T>` - reprezentuje dostawcę, który nie przyjmuje argumentów, ale zwraca obiekt typu T tj. `T get()`.

Przy czym T i R to symbole typów generycznych i można za nie wstawiać dowolne, obiektowe typy Javove, np.:

```
Supplier<Double> randomValue = () -> Math.random();
double value = randomValue.get();
```

lub

```
Function<String, Integer> stringLength = s -> s.length();
int length = stringLength.apply("Programowanie funkcyjne!");
```

W przedstawionym rozwiązaniu nie można oczywiście używać typów prymitywnych, a trzeba korzystać z typów obiektowych m.in. Integer, Double, Boolean. Jednak są też dostępne specjalne interfejsy dla typów prymitywnych tj. IntPredicate, IntConsumer itd.

B. STRUMIENIE

Strumienie w Javie to rozwiązanie, które umożliwia przetwarzanie sekwencji danych w sposób deklaratywny, a co za tym idzie zwięzły. Strumienie pozwalają na wykonywanie operacji takich jak filtrowanie, mapowanie,

redukcje itp. Należy także mieć na uwadze, że strumienie są niemutowalne. Oznacza to, że operacje wykonywane na strumieniach nie zmieniają źródłowej kolekcji danych ani samego strumienia. Zamiast tego, każda operacja strumienia zwraca nowy strumień z wynikami operacji. Nawet w przypadku, gdy zostanie wykonana tylko operacja filtrowania, to wynik będzie nowym strumieniem, który zawiera tylko te elementy, które spełniają warunek filtru, a oryginalny strumień pozostanie niezmieniony. Strumienie oferują szeroki wachlarz operatorów, które można podzielić na operatory pośrednie oraz operatory końcowe.
Operatory pośrednie służą do modyfikowania strumienia i zawsze zwracają nowy strumień, np.:

`filter(Predicate<? super T> predicate)` - filtruje elementy strumienia, pozostawiając tylko te, które spełniają warunek.

Wstawienie w powyższym np. '? super Integer' oznacza, że możemy użyć typu Integer lub dowolnego nadtypu (supertype) dla Integer. W Javie hierarchia dziedziczenia dla klasy Integer wygląda następująco: Integer dziedziczy po Number, a Number dziedziczy po Object.

Operatory końcowe natomiast wykonują operacje na strumieniu i zwracają wynik, np.:

`collect(Collector<? super T, A, R> collector),`

gdzie:

'? super T': oznacza, że kolektor może działać na typie T lub dowolnym nadtypie dla T.
'A': to typ pośredni, którego kolektor używa do gromadzenia elementów (A może być np. ArrayList).
'R': jest typem wynikowym, czyli to, co finalnie dostajemy po zebraniu elementów (dla listy R będzie List<T>, R może być też inną kolekcją np. mapą, zbiorem, a także i pojedynczym obiektem).

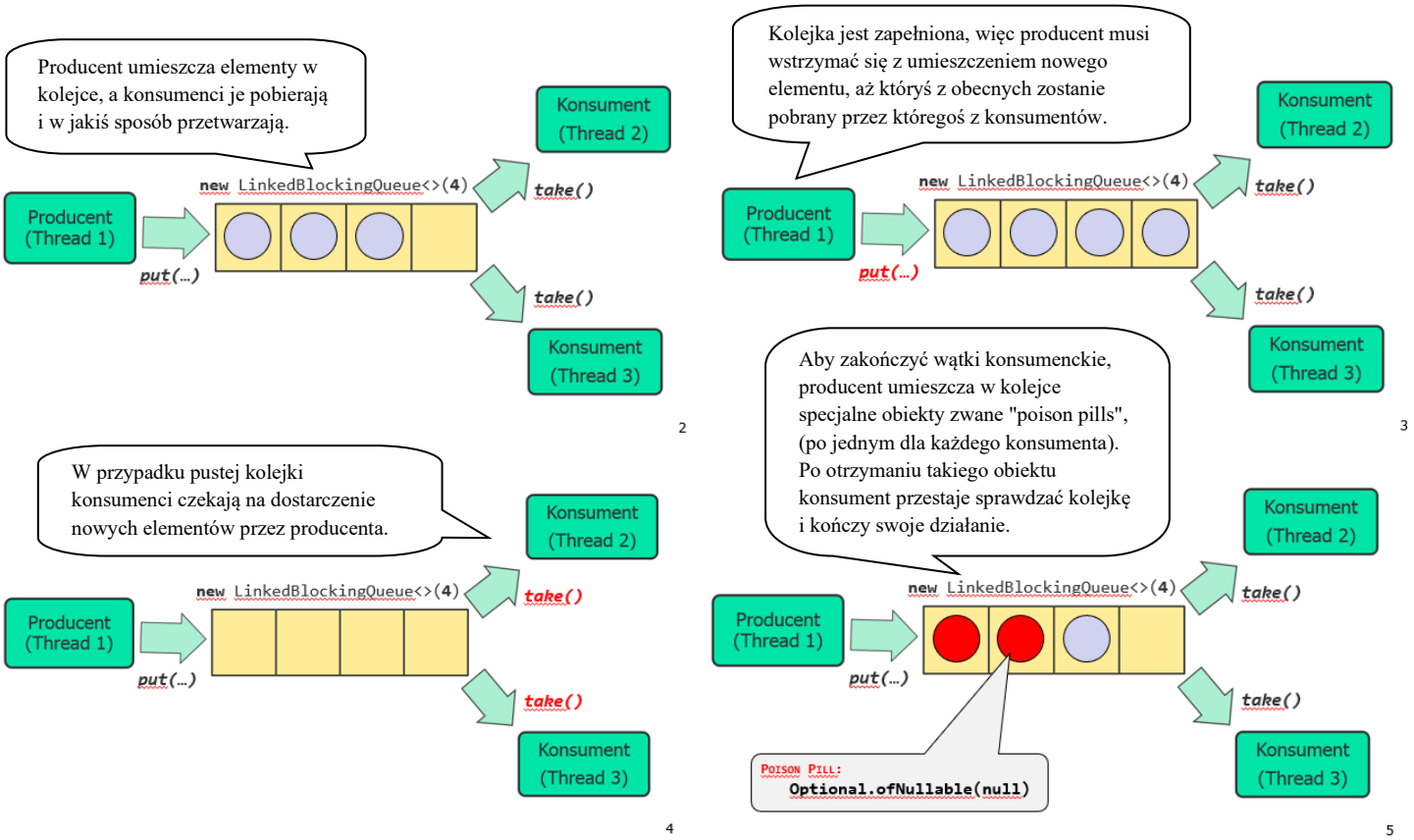
Jak widać, analizowanie dokumentacji może być złożone, jednak korzystanie z tych operatorów jest dość proste, ponieważ w dużej mierze możemy korzystać z wielu gotowych implementacji dostępnych w Javie jak np. *Collectors.toList()*.

```
List<Integer> zbior = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
List<Integer> piatka = zbior.stream()
                        .filter(i -> i.equals(5))//operator pośredni
                        .collect(Collectors.toList());//operator końcowy
```

C. Operator referencji do metody

Podwójny dwukropek (**::**) to operator referencji do metody, pozwala na przekazanie metody jako argumentu do innej metody. Referencje do metody są często używane w połączeniu z API strumieni, aby uprościć i zwiększyć czytelność kodu. Przykładowo - *`System.out::println`* oznacza użycie metody *`println`* obiektu *`System.out`*. Jest to skrótowy zapis, który zastępuje wyrażenie lambda, takie jak np. *`str -> System.out.println(str)`*.

D. Wzorzec Producent – Konsument korzystający z kolejki blokującej



E. Zastosowanie metod isInterrupted i interrupt

Pochodząca z klasy *Thread* metoda *isInterrupted()* służy do sprawdzania, czy bieżący wątek został przerwany. Dokładniej rzecz ujmując, sprawdza tylko stan flagi przerwania wątku, ale jej nie resetuje, nie zmienia jej stanu. Metoda *isInterrupted()* zwraca *true*, jeśli wątek został przerwany, a *false* w przeciwnym przypadku.
W poniższym przykładzie wątek będzie realizował swoje zadania w pętli, dopóki nie zostanie przerwany.

```
Runnable producent = () -> {
        while (!Thread.currentThread().isInterrupted()) {
                // ...
        }
};
```

Jeżeli jednak wątek zostanie obudzony (przerwany podczas snu wymuszonego przez *Thread.sleep(…)*), rzucony będzie wyjątek *InterruptedException*, a flaga przerwania zresetowana. W rezultacie poniższa pętla *while* będzie nadal iterować!

```
Runnable producent = () -> {

        while (!Thread.currentThread().isInterrupted()) {
                // ...
                try {
                        Thread.sleep(1000); // wątek jest uśpiony przez 1 sekundę
                } catch (InterruptedException e) {
                        // Został rzucony wyjątek 'InterruptedException', przez co flaga przerwania
                        // została zresetowana, więc pętla nie zostanie zakończona.
                }
        }
};
```

Można jednak wymusić zakończenie pętli przerywając ponownie wątek w bloku *catch* tj. wywołując metodę *Thread.currentThread().interrupt()*.

```
Runnable producent = () -> {
        while (!Thread.currentThread().isInterrupted()) {
                // ...
                try {
                        Thread.sleep(1000); // wątek jest uśpiony przez 1 sekundę
                } catch (InterruptedException e) {
                        // Został rzucony wyjątek 'InterruptedException', przez co flaga przerwania
                        // została zresetowana. Jednak dzięki poniższemu wywołaniu metody 'interrupt()'
                        // flaga przerwania zostanie ponownie ustawiona, co spowoduje zakończenie pętli.
                        // Przed wywołaniem tej metody można również sprawdzać pewne warunki i ostatecznie
                        // decydować, czy przerwać, czy też zrezygnować z przerwania bieżącego wątku.
                        Thread.currentThread().interrupt();
                }
        }
};
```

F. Mechanizm Files.walkFileTree

Mechanizm *Files.walkFileTree* służy do rekursywnego przechodzenia przez strukturę katalogów i wykonywania operacji na plikach i katalogach. Funkcjonalność ta jest realizowana za pomocą klasy *Files* oraz interfejsu *FileVisitor*. Aby używać *Files.walkFileTree* można skorzystać z dostępnej w Javie klasy *SimpleFileVisitor*, która implementuje interfejs *FileVisitor*. Interfejs ten definiuje cztery metody, ale w zadaniu laboratoryjnym potrzebna będzie tylko metoda *visitFile*. Podczas przechodzenia przez drzewo katalogów metoda ta będzie automatycznie wywoływana dla każdego napotkanego pliku. Poniższy fragment uruchamia mechanizm przeszukiwania wskazanego parametrem *dir* katalogu.

```
Path dir = Paths.get("katalog-testowy");
Files.walkFileTree(dir, new SimpleFileVisitor<Path>() {

        @Override
        public FileVisitResult visitFile(Path path, BasicFileAttributes attrs) throws IOException {
           //...
           return FileVisitResult.CONTINUE;

        }
});
```

2. OPIS PROBLEMU DO ROZWIĄZANIA

Zaimplementuj aplikację, która będzie analizowała zawartość plików znajdujących się we wskazanym katalogu głównym. W zadaniu należy skorzystać z kolejki blokującej np. *LinkedBlockingQueue* (z pakietu *java.util.concurrent*) o ograniczanej parametrem pojemności. Jeden wątek (producent) powinien przechodzić przez drzewo podkatalogów i plików wyszukując pliki tekstowe, a także wstawiać ścieżki na nie wskazujące do kolejki blokującej (ścieżka powinna być obiektem klasy *java.nio.file.Path* opakowanym w *java.util.Optional* np. *Optional<Path> optPath = Optional.ofNullable(path);*). Kolejne dwa wątki (konsumenci) będą pobierały/usuwały obiekty z kolejki i wyszukiwały dziesięć najczęściej występujących wyrazów we wskazanych plikach. W tego typu zadaniu prawdopodobnie wątek producenta szybko wypełni kolejkę obiektami wskazującymi konkretne pliki i zostanie zablokowany do czasu, gdy wątki odbierające elementy wykonają swoją pracę i ponownie opróżnią zapełnioną kolejkę.
Do tworzenia statystyki wyrazów można utworzyć metodę przetwarzająca dane za pomocą strumieni np.

```
private Map<String, Long> getLinkedCountedWords(Path path, Long wordsLimit) throws IOException {
   try (BufferedReader reader = Files.newBufferedReader(path)) {//lub ...(path, StandardCharsets.UTF_8)
      return reader                                            //bez buforowania - Files.readAllLines(path)
         .lines()

         // TODO ...

         .collect(Collectors.toMap(...);
   }
}
```

Zliczanie słów musi być niewrażliwe na wielkość liter i uwzględniać tylko słowa dłuższe od dwóch znaków (znaki specjalne, kropki, przecinki itp. powinny być pomijane). Znalezione słowa powinny być posortowane względem częstotliwości ich występowania w kolejności malejącej i umieszczone w zwracanej mapie typu *LinkedHashMap* (ten typ mapy zachowa kolejność posortowanych danych).
Przykładowy wynik umieszczony w mapie: `{git=224, data=118, version=102, stored=96, ...`

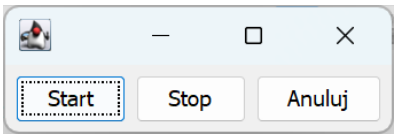
Producent kończąc pracę musi umieścić w kolejce specjalne obiekty tzw. *poison pills*, które przekażą konsumentom informację o konieczności zakończenia ich pracy. Można do tego celu wykorzystać obiekt *Optional.empty()/Optional.ofNullable(**null**)*, poniżej przykładowy fragment kodu konsumenta wykrywający takie obiekty.

```
// WĄTEK KONSUMENTA
while(...){
        ...
   // pobieranie (lub oczekiwanie) obiektu z kolejki
   Optional<Path> optPath = ... ;
        ...
   if(optPath.isPresent()) {
      // przetwarzanie pliku tekstowego ...
      Map<String, Long> countedWords = getLinkedCountedWords(optPath.get(), 10); // dla 10 słów
        ...
   }else{
      //poison pill – trzeba zakończyć wątek
      break;
   }
        ...
}
```

Wszystkie wątki powinny być uruchamiane przy użyciu puli wątków np.

```
   final ExecutorService executor = Executors.newFixedThreadPool(liczbaProducentow + liczbaKonsumentow);
```

Do zatrzymywania producenta można wykorzystać zmienną typu *AtomicBoolean* (z pakietu *java.util.concurrent.atomic*).
Poniżej przedstawiono szkielet programu, na którym można się wzorować. Jednakże realizacja tworzenia statystyk powinna zostać odseparowana od kontrolera aplikacji.




```
package com.lab.statistics;

import java.awt.BorderLayout;
import java.awt.EventQueue;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.FileSystems;
import java.nio.file.FileVisitResult;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.PathMatcher;
import java.nio.file.Paths;
import java.nio.file.SimpleFileVisitor;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.Stream;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.UIManager;
```

```java
public class MainFrame {
      private JFrame frame;
   // ścieżka do katalogu z plikami tekstowymi
   private static final String DIR_PATH = "files";
   // określa ile najczęściej występujących wyrazów bierzemy pod uwagę
   private final int liczbaWyrazowStatystyki;
   private final AtomicBoolean fajrant;
   private final int liczbaProducentow;
   private final int liczbaKonsumentow;
   // pula wątków – obiekt klasy ExecutorService, który zarządza tworzeniem
   // nowych oraz wykonuje 'recykling' zakończonych wątków
   private ExecutorService executor;
   // lista obiektów klasy Future, dzięki którym mamy możliwość nadzoru pracy wątków
   // producenckich np. sprawdzania czy wątek jest aktywny lub jego anulowania/przerywania
   private List<Future<?>> producentFuture;

   public static void main(String[] args) {
      try {
         UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
      } catch (Exception e) {
         e.printStackTrace();
      }
      EventQueue.invokeLater(new Runnable() {
         public void run() {
            try {
               MainFrame window = new MainFrame();
               window.frame.pack();
               window.frame.setAlwaysOnTop(true);
               window.frame.setVisible(true);
            } catch (Exception e) {
               e.printStackTrace();
            }
         }
      });
   }

  public MainFrame() {
      liczbaWyrazowStatystyki = 10;
      fajrant = new AtomicBoolean(false);
      liczbaProducentow = 1;
      liczbaKonsumentow = 2;
      executor = Executors.newFixedThreadPool(liczbaProducentow + liczbaKonsumentow);
      producentFuture = new ArrayList<>();
      initialize();
   }

   /**
    * Initialize the contents of the frame.
    */
   private void initialize() {
      frame = new JFrame();
      frame.addWindowListener(new WindowAdapter() {
         @Override
         public void windowClosing(WindowEvent e) {
            executor.shutdownNow();
         }
      });
      frame.setBounds(100, 100, 450, 300);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      JPanel panel = new JPanel();
      frame.getContentPane().add(panel, BorderLayout.NORTH);
      JButton btnStop = new JButton("Stop");
      btnStop.addActionListener(new ActionListener() {
         public void actionPerformed(ActionEvent e) {
            fajrant.set(true);
            for (Future<?> f : producentFuture) {
               f.cancel(true);
            }
         }
      });
      JButton btnStart = new JButton("Start");
      btnStart.addActionListener(new ActionListener() {
         public void actionPerformed(ActionEvent e) {
            getMultiThreadedStatistics();
         }
      });
      JButton btnZamknij = new JButton("Zamknij");
      btnZamknij.addActionListener(new ActionListener() {
         public void actionPerformed(ActionEvent e) {
            executor.shutdownNow();
            frame.dispose();
         }
      });
```

```java
    panel.add(btnStart);
    panel.add(btnStop);
    panel.add(btnZamknij);
}

 /**
  * Statystyka wyrazów (wzorzec PRODUCENT - KONSUMENT korzystający z kolejki blokującej)
  */
 private void getMultiThreadedStatistics() {
    for (Future<?> f : producentFuture) {
      if (!f.isDone()) {
         JOptionPane.showMessageDialog(frame, "Nie można uruchomić nowego zadania!
                                       Przynajmniej jeden producent nadal działa!", "OSTRZEŻENIE", JOptionPane.WARNING_MESSAGE);
         return;
      }
    }

    fajrant.set(false);
    producentFuture.clear();
    final BlockingQueue<Optional<Path>> kolejka = new LinkedBlockingQueue<>(liczbaKonsumentow);
    final int przerwa = 60;


    Runnable producent = () -> {
       final String name = Thread.currentThread().getName();
       String info = String.format("PRODUCENT %s URUCHOMIONY ...", name);
       System.out.println(info);

      while (!Thread.currentThread().isInterrupted()) {
            if(fajrant.get()) {

                // TODO przekazanie poison pills (kolejka.put(Optional.empty());) konsumentom i zakończenia działania
            } else {

                // TODO Wyszukiwanie plików *.txt i wstawianie do kolejki ścieżki opakowanej w Optional (Optional<Path>
                // optPath = Optional.ofNullable(path); kolejka.put(optPath);) lub oczekiwanie jeśli kolejka
                // pełna. Do wyszukiwania plików można użyć metody Files.walkFileTree oraz klasy SimpleFileVisitor<Path>
            }
            info = String.format("Producent %s ponownie sprawdzi katalogi za %d sekund", name, przerwa);
            System.out.println(info);
            try {
               TimeUnit.SECONDS.sleep(przerwa);
            } catch (InterruptedException e) {
               info = String.format("Przerwa producenta %s przerwana!", name);
               System.out.println(info);
               if(!fajrant.get()) Thread.currentThread().interrupt();
            }
      }
      info = String.format("PRODUCENT %s SKOŃCZYŁ PRACĘ", name);
      System.out.println(info);
   };


    Runnable konsument = () -> {
       final String name = Thread.currentThread().getName();
       String info = String.format("KONSUMENT %s URUCHOMIONY ...", name);
       System.out.println(info);

       while (!Thread.currentThread().isInterrupted()) {
          try {

             // TODO pobieranie ścieżki (Optional<Path> optPath = kolejka.take();) i tworzenie statystyki wyrazów
             // lub oczekiwanie jeśli kolejka jest pusta. Brak ścieżki tj. !optPath.isPresent() oznacza koniec pracy
         } catch (InterruptedException e) {
            info = String.format("Oczekiwanie konsumenta %s na nowy element z kolejki przerwane!", name);
            System.out.println(info);
            Thread.currentThread().interrupt();
         }
      }
      info = String.format("KONSUMENT %s ZAKOŃCZYŁ PRACĘ", name);
      System.out.println(info);
   };

   //uruchamianie wszystkich wątków-producentów
   for (int i = 0; i < liczbaProducentow; i++) {
      Future<?> pf = executor.submit(producent);
      producentFuture.add(pf);
   }
   //uruchamianie wszystkich wątków-konsumentów
   for (int i = 0; i < liczbaKonsumentow; i++) {
      executor.execute(konsument);
   }
 }
```

```java
/** Metoda zwraca najczęściej występujące słowa (ich liczbę określa wordsLimit, a słowa są sortowane względem
 * częstotliwości ich występowania) we wskazanym pliku tekstowym
 */
private Map<String, Long> getLinkedCountedWords(Path path, int wordsLimit) {
   //konstrukcja 'try-with-resources' - z automatycznym zamykaniem strumienia/źródła danych
   try (BufferedReader reader = Files.newBufferedReader(path)) {// wersja ze wskazaniem kodowania
                                                               // Files.newBufferedReader(path, StandardCharsets.UTF_8)
        return reader.lines() // można też bez buforowania - Files.readAllLines(path)
                              // do drukowania elementów strumienia można użyć .peek(System.out::println)
         //TODO
         // 1. Podział linii na słowa, można skorzystać z funkcji map, metody split i wyrażenia regularnego np. "\\s+"
         //    oznaczającego jeden lub więcej tzw. białych znaków. Po tej operacji pewnie będzie potrzebna konwersja
         //    strumienia 'String[]' do strumienia 'String' za pomocą .flatMap(Arrays::stream)
         // 2. Wycięcie wszystkich znaków, które nie tworzą słów m.in. ;,.?!:
         //    Można użyć np. .map(word -> word.replaceAll("[^a-zA-Z0-9ąęóśćżńźĄĘÓŚĆŻŃŹ]{1}", ""))
         // 3. Filtrowanie słów - tylko z przynajmniej trzema znakami, użyj funkcji filter, metody matches i wyrażenia
         //    regularnego w lambdzie np. word -> word.matches("[a-zA-Z0-9ąęóśćżńźĄĘÓŚĆŻŃŹ]{3,}")
         //    (bez uwzględniania polskich znaków byłoby "[a-zA-Z]{3,}")
         // 4. Konwersja do małych liter, aby porównywanie słów było niewrażliwe na wielkość liter (można zrobić wcześniej,
         //    przed p. 2, wtedy wyrażenia regularne nie będą musiały uwzględniać wielkich literek)
         // 5. Grupowanie słów względem liczebności ich występowania, można użyć
         //    funkcji collect z Collectors.groupingBy(Function.identity(), Collectors.counting()),
         //    po tej operacji należy zrobić konwersję na strumień tj. .entrySet().stream()
         // 6. Sortowanie względem przechowywanych w mapie wartości, w kolejności malejącej,
         //    można użyć Map.Entry.comparingByValue(Comparator.reverseOrder())
         // 7. Ograniczenie liczby słów do wartości z wordsLimit
          .collect(Collectors.toMap( //umieszczenie elementów strumienia w mapie zachowującej kolejność tj. LinkedHashMap
                Map.Entry::getKey,
                Map.Entry::getValue,
                 (k,v) -> { throw new IllegalStateException(String.format("Błąd! Duplikat klucza %s.", k)); },
                 LinkedHashMap::new));

   }catch (IOException e) {throw new  RuntimeException(e);}
}
```

Zadanie dodatkowe

Stwórz wersję programu, który zamiast statystyk wyrazów będzie wyznaczał podobieństwo (korzystając z tzw. miary kosinusowej) znalezionych dokumentów tekstowych do jednego wybranego/wskazanego dokumentu wzorcowego.

Literatura

- Herbert Schildt, *Java. Kompendium programisty. Wydanie X*, 2018, Helion